Advanced Automation Group Project

2ºPeriod of the 1º Semester - 2025/2026

CMAPSS Jet Engine Simulated Data



Prof. Susana Vieira

Prof. Bernardo Chagas

Prof. Diogo Oliveira

Prof. Felipe Santos

Number	Name
116011	David Estevas
116086	Duarte Nogueira
117164	Mathias Dam

Index

Introduction 3
Problem clarification and project goals: 3
CMAPSS 3
Data processing 4
Linear Regression 5
Random Forest 6
Support Vector Regression 7
Long Short-Term Memory 8
Result comparison and conclusions 9
Bibliografia 10

Table Index

Figure 1 - Column name definition 4
Figure 2 - Search and elimination of useless features 4
Figure 3 - Data processing info 5
Figure 5 - Prediction of RUL 6
Figure 6 - RUL clipping 6
Figure 7 - Training and predtiction of the random forest model 7
Figure 8 – Scaling of SVR data. 7
Figure 9 - Limiting amount of SVR data. 8
Figure 10 - Scaling LSTM Data 9
Figure 11 – Structure of the LTSM Model 9

Table Index

Table 1 - Top 10 parameters correlated with RUL 5
Table 2 – Performance metrics of the linear regression model 6
Table 3 - Performance metrics of the random forest model 7
Table 4 - Performance metrics of the support vector regression model 8
Table 5 – Performance metrics of the Long Short-Term Memory 9
Table 6 - Performance comparison between all the models 10

Equation Index

Equation 1 - RSME equation 4
Equation 2 - Linear regression 6
Equation 3 – RSME equation 9
Equation 4 - R^2 equation 10

Introduction

This group project was developed within the scope of the Advanced Automation course unit during the second period of the first semester of the 2025/2026 academic year. The goal of the project is to use the CMAPSS dataset to train a model to predict the remaining useful life cycles of a jet engine before it breaks down. This is useful for example for doing predictive maintenance, as the motor can be sent to maintenance whenever the model predicts a low number of remaining useful cycles. To make this model, multiple approaches were implemented and compared.

Problem clarification and project goals:

The CMAPSS Jet Engine Simulated Data is a dataset containing run-to-failure measurements from multiple turbofan engines operating under varying conditions generated using the Commercial Modular Aero-Propulsion System Simulation (CMAPSS). Each engine is monitored over successive operating cycles using a set of sensor measurements that measure its internal condition as it gradually degrades. Since the exact failure point is known for each engine, every cycle can be labelled with its Remaining Useful Life (RUL), defined as the number of cycles remaining before failure.

The objective of this project is to develop and compare machine learning models capable of predicting the Remaining Useful Life of jet engines based on sensor data. Accurate RUL predictions enable predictive maintenance strategies, allowing engines to be serviced before failure while avoiding unnecessary early maintenance. This is particularly important in safety-critical and cost-sensitive applications such as aviation.

Several modelling approaches were implemented and tested. In addition, data preprocessing steps such as normalization, sequence generation, and label construction were included. Model performance is evaluated using metrics to identify the most effective approach for RUL prediction on the CMAPSS dataset.

CMAPSS

The CMAPSS contains 4 different datasets:

- FD001: contains data from engines that failed because of degradation of the High-Pressure Compressor. The engines were operated in ambient conditions designed by “ONE” and the data is already spitted in training and test sets both containing data from 100 different engines.
- FD002: contains data from engines that failed because of degradation of the High-Pressure Compressor. The engines were operated in ambient conditions designed by “SIX” and the data is already spitted in training and test sets, containing data from 260 and 259 different engines respectively.
- FD003: contains data from engines that failed because of degradation of the High-Pressure Compressor or degradation of the Fan. The engines were operated in ambient conditions designed by “ONE” and the data is already spitted in training and test sets both containing data from 100 different engines.
- FD004: contains data from engines that failed because of degradation of the High-Pressure Compressor or degradation of the Fan. The engines were operated in ambient

conditions designed by “SIX” and the data is already spitted in training and test sets, containing data from 248 and 249 different engines respectively.

The engine is operating normally at the start of each time series, and develops a fault at some point during the series. In the training set, the fault grows in magnitude until system failure. In the test set, the time series ends some time prior to system failure. All the datasets contain a file with actual Remaining Useful Life for the test set data [1].

Data processing

In order to build a proper model to analyse the data, we need to organize and clean the C-MAPSS dataset.

The first step was organising the data into a table. The columns of the table were defined by the engine ID (unit_nr), the number of completed working cycles (time_cycles), the engine settings (setting_1, setting_2, setting_3) and all the readings of the 22 sensors (s_1, … , s_22):

```
# Column names (C-MAPSS Standard)
index_names = ['unit_nr', 'time_cycles']
setting_names = ['setting_1', 'setting_2', 'setting_3']
sensor_names = ['s_{}'.format(i) for i in range(1, 22)]
col_names = index_names + setting_names + sensor_names
```

Figure 1 - Column name definition

Now that it’s properly organized, the datasets are able to be processed. The raw files were read and then organized as shown previously, in addition to the training set, were calculated the Remaining Useful Life (RUL). RUL for any given engine unit i at time cycle t is defined as the difference between the maximum cycle life of that unit ($T_{max,i}$) and the current time cycle:

$$RUL_i(t) = T_{max,i} - t$$

Equation 1 - RSME equation

To achieve better results, reducing noise is essential. For that we must find the columns with standard deviation equalling 0 in the training sets and then drop them. Having a column with 0 standard deviation means that the parameter doesn’t give useful information regarding the analysis of the evolution of the wear in the engines. In order to maintain the consistency in the datasets, the same columns removed in the training set will be removed as well in the test set.

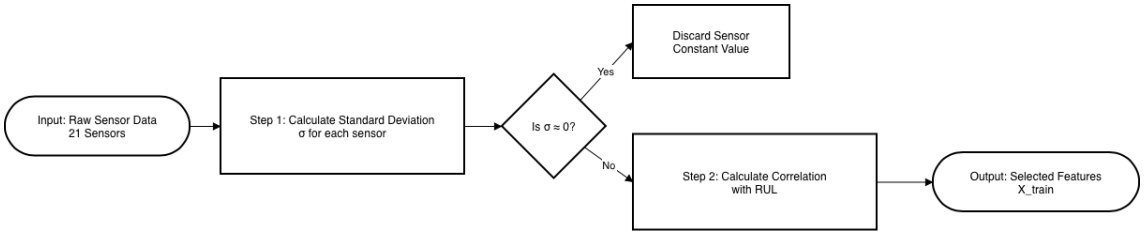


Figure 2 - Search and elimination of useless features

This resulted on the remotion of the of the columns “setting_3”, “s_18” and “s_19” for the datasets FD001 and FD003.

```
========== Processing FD001 ==========
Data loaded: Train (20631, 26), Test (13096, 26)
-> Removing 3 constant columns: ['setting_3', 's_18', 's_19']
Success! Files saved in: ../data/processed

========== Processing FD002 ==========
Data loaded: Train (53759, 26), Test (33991, 26)
-> No constant columns found.
Success! Files saved in: ../data/processed

========== Processing FD003 ==========
Data loaded: Train (24720, 26), Test (16596, 26)
-> Removing 3 constant columns: ['setting_3', 's_18', 's_19']
Success! Files saved in: ../data/processed

========== Processing FD004 ==========
Data loaded: Train (61249, 26), Test (41214, 26)
-> No constant columns found.
Success! Files saved in: ../data/processed

--- Processing Complete ---
```

Figure 3 - Data processing info

Afterwards, the processed files are going to be stored in a different folder alongside all the other processed datasets.

Once stored, the training set data needs to be loaded to proceed with an analysis to find correlations between the different parameters. By finding the sensors with the highest correlation to the RUL it's possible to understand which factors have more impact. This could prove to be helpful when building a model, for feature selection, reducing the problem's complexity leading to simpler, more accurate and interpretable model. Analysing the correlation chart between the features, these were the 10 features with highest correlation with RUL for the 4 different datasets:

Table 1 - Top 10 parameters correlated with RUL

FD001		FD002		FD003		FD004	
Feature	Correlation	Feature	Correlation	Feature	Correlation	Feature	Correlation
time_cycles	0.736241	time_cycles	0.733383	s_11	0.688915	time_cycles	0.610620
s_11	0.696228	s_16	0.071352	s_4	0.657224	s_14	0.078126
s_4	0.678948	s_11	0.046950	s_13	0.656340	s_11	0.056639
s_12	0.671983	s_14	0.042325	s_8	0.655345	s_16	0.053804
s_7	0.657223	s_4	0.040978	s_17	0.649220	s_4	0.045881
s_15	0.642667	s_15	0.038455	s_3	0.624236	s_17	0.032939
s_21	0.635662	s_17	0.027031	s_2	0.590564	s_3	0.032924
s_20	0.629428	s_3	0.026942	s_9	0.551843	s_9	0.024727
s_2	0.606484	s_9	0.015306	time_cycles	0.482744	s_10	0.008924
s_17	0.606154	unit_nr	0.014608	s_14	0.481807	s_2	0.004443

Linear Regression

To build a linear regression model, we need to load the processed data and train the model to predict the RUL of an engine, and then compare it with the actual RUL.

The first thing to do is to only train the model based on sensor values, since those are the only data values obtained from the engine's functioning. For that, when defining the model's features, the values from the "unit_nr", "time_cycles" and "RUL" columns are ignored. The Linear Regression model attempts to approximate the RUL function y as a linear combination of the selected sensor features $x_1, x_2, \ldots, x_n$

$$\hat{y} = \beta_0 + \beta_{1x_1} + \beta_{2x_2} + \ldots + \beta_{nx_n} + \epsilon$$

Equation 2 - Linear regression

Where β represent the coefficients learned during training via the Least Squares method. After training the model, the next step is to evaluate it on the test set. The RUL is predicted at the last available cycle in each test set and compared with the actual RUL values in the .txt file from the respective dataset. To evaluate the performance of the model, these were the values of RMSE and R^2 for linear regressions on all 4 datasets:

Table 2 – Performance metrics of the linear regression model

Dataset	RMSE	R^2
FD001	32.04	0.4055
FD002	33.94	0.6016
FD003	57.08	-0.9011
FD004	47.76	0.2328

```
test_last_cycle = df_test.groupby('unit_nr').last().reset_index()
X_test = test_last_cycle[features]

y_pred = model.predict(X_test)
y_true = df_rul['RUL_actual']
```

Figure 4 - Prediction of RUL

Random Forest

To build a random forest model we need to load again the processed data and train it properly to predict the RUL of an engine and compare it with the actual RUL value.

To avoid problems related to overfitting the RUL values were clipped. Since the values of the sensors barely change with hight RUL values, meaning that the wear is barely noticeable, it's useless to train it for those values because it may introduce some noise and make the model learn some inexistent patterns leading to worse performance. For that matter, both predicted and actual RUL values were clipped at a maximum of 125 cycles.

```
df_train['RUL'] = df_train['RUL'].clip(upper=125)
y_true = df_rul['RUL_actual'].clip(upper=125)
```

Figure 5 - RUL clipping

Similarly to the linear regression, the "unit_nr", "time_cycles" and "RUL" columns were ignored and only the sensor readings were taking into accountability.

The following steps are defining the model's hyperparameters, training and testing. Similarly to the linear regression model, the predicted values of RUL will be compared with the real values stored in the RUL .txt file of the respective dataset.

```
model = RandomForestRegressor(n_estimators=100, max_depth=12, random_state=42, n_jobs=-1)

print(f"-> Training {file_id}...")
model.fit(X_train, y_train)

test_last_cycle = df_test.groupby('unit_nr').last().reset_index()
X_test = test_last_cycle[features]
y_pred = model.predict(X_test)
```

Figure 6 - Training and predtiction of the random forest model

The selected values for the hyperparameters were:

- **n_estimators=100** – Defines the number of decision trees to 100, giving the model robustness while not overloading the machine with pointless processing since from a certain number of trees the error tends to stabilize.
- **max_depth**=12 – Defines the maximum depth of a single tree. This value give the tree enough depth to learn degradation regimes and non-linear patterns while avoiding overfitting.
- **random_state=42** – Ensures that the results are reproducible.
- **n_jobs=-1** – Makes the machine use all the available cores, resulting in faster training times.

Computing the same metrics used previously, these were the results obtained for the random forest model for the 4 datasets:

Table 3 - Performance metrics of the random forest model

Dataset	RMSE	R^2
FD001	17.21	0.8155
FD002	17.98	0.8247
FD003	20.08	0.7373
FD004	20.28	0.7775

Support Vector Regression

In s similar way to the previous models, an implementation for support vector regression (SVR) was made.

SVR is sensitive to the scaling of the training data, so first, all the training and test data was normalized using the sklearn.preprocessing StandardScaler() class.

```
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
```

Figure 7 – Scaling of SVR data.

The time it takes SVR to execute can increase a lot as the amount of training data increases. Therefore, the amount of training data was limited to 10000 datapoints. The 10000 datapoints are selected at random. This is because the data most likely contains trends, and we want to avoid only training on a select chunk of the data that has some trend not present across all the data.

```
# Downsampling for speed
limit = 10000
n = len(X_train_s)

if n > limit:
    idx = np.random.choice(n, size=limit, replace=False)
    X_train_s = X_train_s[idx]
    y_train = y_train[idx]
```

Figure 8 - Limiting amount of SVR data.

After that, the model was trained using the scikit-learn SVR model. The hyperparameters were selected as:

- **Kernel: "RBF"**. This was chosen because the relationship between engine sensor readings and RUL is assumed to be highly non-linear. Linear or polynomial models struggle to capture degradation patterns, while the RBF kernel can model smooth, complex trends without requiring explicit feature engineering.
- **C: 10**. C controls the trade-off between model simplicity and fitting accuracy. A moderate value like 10 allows the model to fit the data reasonably well while still applying regularization to avoid overfitting. Very large values of C tend to overfit, while very small values underfit.
- **Epsilon: 5**. Epsilon defines the width of the error-insensitive zone. The number 5 was selected because the RUL values are usually in the range of 50-150. Therefore, an error of 5 is determined as "acceptable".

It is worth to mention that the hyperparameters were mostly chosen based on "guessing", and a more sophisticated approach, for example using a grid search, is likely to yield better results.

Comparing the training and test errors and plotting an overfit curve to detect if the model is overfitting should also be done to better determine C - If there is evidence of overfitting, C can be reduced. This was however not conducted due to time constraints.

Training the model with the hyperparameters specified above, this was the result:

Table 4 - Performance metrics of the support vector regression model

Dataset	RMSE	R^2
FD001	18.56	0.7854
FD002	30.01	0.5116
FD003	20.46	0.7271
FD004	35.75	0.3082

Long Short-Term Memory

In a similar way to the previous models, an implementation for a Long Short-Term Memory (LSTM) network was made. The key difference compared to the classical ML models is that an LSTM does not learn from independent datapoints, but from sequences of datapoints, which makes it well suited for degradation problems where the trend over time matters, not only the current sensor values.

Because neural networks are sensitive to the scale of the inputs, the sensor features are normalized using MinMaxScaler.

```
scaler = MinMaxScaler()
df_train[features] = scaler.fit_transform(df_train[features])
df_test[features] = scaler.transform(df_test[features])
```

Figure 9 - Scaling LSTM Data

Then, the data is divided into sequences with a length of 30 cycles using the "gen_seq" function. This means that each training sample will consist of the sensor readings from 30 consecutive timesteps, and the model will learn to predict the Remaining Useful Life (RUL) associated with the end of that 30-cycle window.

Before training, the target values are scaled by dividing by 125 (y_train = y_train / 125.0). This is a simple form of target normalization, so the model learns to predict values in a smaller numeric range, which typically makes optimization more stable. After prediction, the output is scaled back to the original RUL units by multiplying by 125.

After that, a LTSM model is created using the "create_ltsm_model" function. This model uses the Keras library to define the structure of the model. Dropout is used after each layer to prevent overfitting.

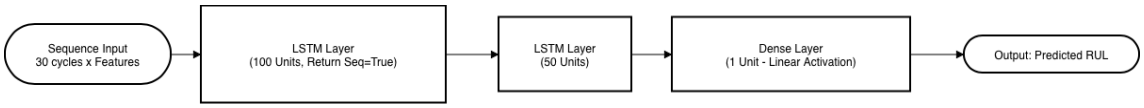


Figure 10 – Structure of the LTSM Model

The following hyperparameters were chosen:

- Number of layers: 2.
- Layer sizes: 100 and 50
- Dropout rate: 0.2

These hyperparameters were mostly chosen based on intuition and what hyperparameters are common for similar problems. A more systematic approach testing out different parameters and plotting the overfit curve could yield better results.

Table 5 – Performance metrics of the Long Short-Term Memory

Dataset	RMSE	R^2
FD001	14.49	0.8692
FD002	16.61	0.8491
FD003	13.77	0.8763
FD004	20.91	0.7607

Result comparison and conclusions

To get an idea of the model's performance, the chosen metrics were the Root Mean Squared Error (RSME) and R^2.

RSME in this scenario reflects the average error (in cycles) committed by the model on the prediction of the RUL of all the engine ID's, it's also a metric that penalizes higher errors.

$$RSME = \sqrt{\frac{1}{nº\ of\ Engines}\sum\left(RUL_{actual} - RUL_{predicted}\right)^2}$$

Equation 3 – RSME equation

The R^2 measures the relation between the total sum of the squares and the residual some of squares, meaning that this metric tells us what proportion of the model's variance can be explained by the independent variables.

$$R^2 = 1 - \frac{\sum\left(RUL_{actual} - RUL_{predicted}\right)^2}{\sum\left(RUL_{actual} - RUL_{avg\ actual}\right)^2}$$

Equation 4 - R^2 equation

For better visualisation, the next table contain all the results obtained from the 4 different models:

Table 6 - Performance comparison between all the models

Dataset	Linear Regression		Random Forest		Support Vector Regression		Long Short-Term Memory	
	RMSE	R^2	RMSE	R^2	RMSE	R^2	RMSE	R^2
FD001	32.04	0.4055	17.21	0.8155	18.56	0.7854	14.49	0.8692
FD002	33.94	0.6016	17.98	0.8247	30.01	0.5116	16.61	0.8491
FD003	57.08	-0.9011	20.08	0.7373	20.46	0.7271	13.77	0.8763
FD004	47.76	0.2328	20.28	0.7775	35.75	0.3082	20.91	0.7607

Linear Regression: as shown in results, linear regression is by far the worst option from the 4 models to solve this problem. The results suggest that the engine wear (sensor readings) evolution throughout the time follow a non-linear tendency, being that the main reason why the model performs so poorly.

Support Vector Regression: this model presents decent results for the datasets FD001 and FD003 but struggles with FD002 e FD004. The model seems to struggle with larger datasets, as it induces more noise and variance.

Random Forest: the results for the random forest model show a great balance between performance and robustness. This model shows great capability handling larger and more complex datasets, inclusively obtaining the best performance for the dataset FD004.

Long Short-Term Memory: this model has shown the best overall performance. This type of neural networks is ideal for time series forecasting, and since the engine wear is directly related with time, makes this model the best option to predict the RUL. It's worth noting that in the FD004 dataset the performance is pretty similar to the random forest model, that happens due to the dataset's complexity which introduces noise. That shows that the model is sensitive noise.

Bibliografia

[1] NASA, "CMAPSS Jet Engine Simulated Dataset - NASA Open Data Portal," 31 March 2025. [Online]. Available: https://data.nasa.gov/dataset/cmapss-jet-engine-simulated-data. [Accessed on December 12, 2025]